

CODEHIVE

JAVA PROGRAMMING: MULTI-WAY BRANCHING

► Introduction to Switch

Instead of writing many **if...else if** statements, Java provides the **switch statement**. It is cleaner and more efficient for checking a single variable against multiple fixed values.

Real-world Logic: "Elevator buttons: Press 4 to go to Floor 4."

► Basic Syntax

```
switch(expression) {  
    case value1:  
        // code block  
        break;  
    case value2:  
        // code block  
        break;  
    default:  
        // runs if no cases match  
}
```

► The Importance of 'break'

Without the `break` keyword, Java will continue executing all subsequent cases regardless of whether they match. This is called **Fall-through**.

```
int day = 2;
switch (day) {
    case 1: System.out.println("Mon");
    case 2: System.out.println("Tue"); // Matches
    case 3: System.out.println("Wed"); // Runs due to fall-through!
}
```

► Switch with Multi-Case Grouping

Multiple cases can share the same execution block for cleaner logic.

```
switch (month) {
    case 12: case 1: case 2:
        System.out.println("Winter");
        break;
    case 3: case 4: case 5:
        System.out.println("Spring");
        break;
}
```

► Switch vs. If-Else

- **Use Switch:** When comparing one variable against several fixed values (integers, strings, or chars).
- **Use If-Else:** When dealing with ranges (e.g., `score > 90`) or complex boolean logic.

► Decision Efficiency Summary

- **break:** Essential to exit the switch block once a match is handled.
- **default:** The safety net that runs when no other case is valid.
- **Supported Types:** byte, short, char, int, String, and Enums.
- **Cleanliness:** Significantly reduces "Boilerplate" code in large decision trees.